1 Preparacion para el TP2

1.1 Estructuras elegidas

1. Usuarios — ABB para la busqueda normal y HEAP para obtener el ranking por millas (Esto abarcaría crearUsuario, RegistrarUsuario y RegistrarUsuarioConReferidor). El arbol del usuario va a estar compuesto por nodos, dichos nodo contendrán el ID del usuario, su categoría, su saldoDisponible (Es decir las millas disponibles) y tambien las totales, esto para aprovechar tambien y obtener instantamentamente su categoría. — CONSULTAR— Para la parte de paresExclusivosDeTransferidores, podria quizas implementarse otro arbol que contenga la los usuarios que a los que se le hizo la transferencia, ademas de una constante que diga cuantos elementos tiene el árbol (La idea detrás de esto es que si el arbol tiene +1 hijo es porque entonces no son transferidores exclusivos. Pero si tiene exactamente un hijo hay que buscar nuevamente sobre el arbol de usuarios, o con un handle sería mucho más eficiente y verificar las transferencias de dicho usuario nuevamente, si su arbol de transferidores vario es porque no son paresTransferidores, si son iguales entonces si.) ¿Es esta idea equivalente con otra estructura? Suena un poco raro crear un arbol cuando solo interesa recorrerlo si tiene un elemento. Para eso no sería mejor crear una lista o algo similar?

2. Transacciones — ArrayList(Transaccion): (Abarca los procs CanjearMillas,TranferirMillas, AcumularMillas) (Creo que esto no es necesario, porque en realidad no se utiliza, se utilizaría capaz en ResumenTransacciones, pero en la consigna del tp no hay que implementarlo asi que este quizas no va y directamente iria todo dentro del arbol de usuarios.)

3. Promociones — DiccionarioDigital (TRIE): como clave tiene el nombre de la promocion, y como valor al objeto Promocion. (Abarca los procs IniciarPromocion,IniciarPromocionConTope y FinalizarPromocion)

4. ElMasAcumulador: Heap(HandleUsuario) ElMasAcumulador es un atributo de la clase airmiles cuya estructura Heap que contiene el handle del usuario, es decir, una referencia abstracta de los usuarios es mucho mas fácil acceder al Objeto usuario en tiempo O(1). A su vez la clase Heap implementa la interfaz comparator (PorMasAcumulado o algo similar) que se encarga de chequear que para todo objeto del heap este primero siempre el que mas millas acumuló.

5. elMasCanjeador: Heap(HandleUsuario) Misma idea con el mas canjeador, tiene un handler para acceder a la referencia del usuario en O(1) y el heap implementa el mismo comparator pero con un CompareTo pensado para la persona que mas millas canjeo.

6. rankingMillas: Heap(HandleUsuarios), esto entiendo que costaría O(N log U) Obtener los usuarios es O(1) en el heap y reordenar el heap (sift-down) es O(log U). El total queda entonces O(N log U) para obtener los usuarios y reordenar todo el heap, la diferencia entre este y los dos anteriores es que aca hay que reordenar el heap y eso cuesta O(N log U). Ademas, esta clase del heap utiliza otro comparator, que compara justamente por el que tiene mas millas. Creo que igual aca hay que tener un cuidado extra. Estamos SACANDO los elementos del Heap, si luego no los vuelvo a insertar, no se rompe el invariante pero se puede romper el sistema. Esto lo podemos consultar.

1.2 Objetos que vamos a usar (No esta todo completo, es un pantallazo)

1. Usuario: tiene un ID, una categoria, las millas disponibles y las millas totales

2. Promocion: tiene un nombre, un factor y millas restantes. Al crearlo se debe especificar si es con o sin tope

3. Clase Transaccion:
   Enum TipoDeTransaccion (Canjeo, transferencia, referido)
   int UsuarioOrigen
   int UsuarioDestino
   float distancia
   floatj millas
   str NombrePromo
   float factorPromo

   Hay que definir tambien aca los métodos del objeto transaccion (tener solo una clase con atributos, lo vuelve una clase anémica. No tiene mucho sentido. Otra opcion sino sería usar una estructura como un struct o algo asi aunque no la vimos.)

4. Objeto Promocion
   str NombrePromocion
   float FactorPromocion
   bool TieneTope
   float MillasRestantes

   (Creo que los métodos se definen con un override porque realmente quien los implementa es la clase Airmiles que pisa los metodos del objeto promoción)
   metodos:
   IniciarPromocion
   IniciarPromocionConTope
   FinalizarPromocion

1.3 Procedimientos / Funciones de AirMiles

1. IniciarSistema O(1) Solo inicializaciones.

2. RegistrarUsuario / RegistrarUsuarioConReferidor O(log U) en el caso promedio porque es meter al usuario en un arbol binario y O(U) en el peor caso si el arbol esta degenerado, es decir no esta balanceado (esta es la justificacion además de por que podemos utilizar un ABB en vez de un AVL.)

3. Canjear Millas, Transferir Millas: tardan O(log U) porque buscar al usuario en el arbol cuesta eso, y asignarle las millas es O(1)

4. Acumular Millas: tiene complejidad O(log U + $|L|$) porque la busqueda del usuario es log U y la busqueda de la promocion es $|L|$ ya que su estructura es un TRIE

5. elMasAcumulador / elMasCanjeador O(1) Porque la implementacion del Heap para obtener la raiz es instantánea.

6. rankingMillas: O(N log U) porque sacar la usuario es O(1) y eso se hace N veces, y luego reordenar el heap es O(Log U).

7. paresTransferidoresExclusivos: O(U) porque en el peor de los casos hay que buscar al usuario en el arbol (Faltaría entrar mejor en detalles con esto, consultar!)